Roppi

A game of ropes for learning sports-climbing knots

Built with weft, under Literate Driven Development

October 19, 2026

Contents

1 The domain: a game of ropes for climbing knots 3
- 1.1 Purpose . . . 3
- 1.2 The problem . . . 3
- 1.3 Scope . . . 3
- 1.4 Who it is for . . . 4

2 Domain analysis 5
- 2.1 Concepts . . . 5
- 2.2 Invariants . . . 5
- 2.3 Closure of open ends . . . 6
- 2.4 Decided knot choices . . . 6
- 2.5 Vocabulary . . . 6

3 Architecture 8
- 3.1 Decision: rendering in three.js . . . 8
- 3.2 Decision: the rope is a Rapier chain, drawn as one tube . . . 8
- 3.3 Decision: closure is a point behind the background . . . 10
- 3.4 Decision: recognition by planar projection and invariants . . . 10
- 3.5 Decision: anchors are closed components . . . 10
- 3.6 Decision: lessons are data . . . 10
- 3.7 Decision: literate source with weft . . . 11
- 3.8 Decision: tooling . . . 11
- 3.9 Forbidden edges . . . 11
- 3.10 Named seams . . . 12

4 Development stages 13
- 4.1 Stage 1 — the walking skeleton . . . 13
- 4.2 Stage 2 — the recogniser . . . 14
- 4.3 Stage 3 — guided mode for one knot . . . 14
- 4.4 Stage 4 — sandbox . . . 14
- 4.5 Stage 5 — quiz mode . . . 14
- 4.6 Stage 6 — the knot catalogue . . . 14
- 4.7 State of the work . . . 15

5 Scaffold **16**
5.1 Packages . . . 16
5.2 Build and test configuration . . . 17
5.3 Page shell . . . 18

6 Rope model **20**
6.1 Shapes the rest of the code reads . . . 20
6.2 Physics model . . . 21
6.3 Tests . . . 23

7 Topology **25**
7.1 Closure . . . 25
7.2 Crossings and the Fox matrix . . . 26
7.3 Determinant . . . 28
7.4 Recogniser . . . 29
7.5 Tests . . . 30

8 Renderer **32**
8.1 The tube . . . 32
8.2 Tests . . . 34

9 Lessons and the page **36**
9.1 Lesson record . . . 36
9.2 Anchor . . . 37
9.3 Rope stage . . . 37
9.4 Application . . . 41

10 Assembly **43**
10.1 Tooling and page . . . 43
10.2 Rope model . . . 44
10.3 Topology . . . 45
10.4 Renderer . . . 46

Chapter 1

The domain: a game of ropes for climbing knots

Roppi is a browser game for learning the knots of sports climbing. A player sees a rope drawn in three dimensions, grabs it with the mouse, and moves it. The shapes the rope takes are the knots the player is learning to recognise and tie. The player is a complete beginner.

1.1 Purpose

Roppi teaches climbing knots through hands-on manipulation, not memorised diagrams. A knot is a shape the rope holds, and the shape changes only when the rope is moved. Each knot is connected to its job in the climbing process: why it is tied at that point, and what a mistake looks like.

1.2 The problem

Climbing knots are usually learned from diagrams and from an instructor's hands. A diagram shows one picture of a knot and says nothing about how it is built up or how it can go wrong. A beginner needs to see the rope move through the crossings that make the knot, and to be told when the knot they have built is the one they meant to build. Roppi makes the crossings visible and checks the result.

"Solved" means that a beginner can build each lesson's knot from nothing, and the game names the knot the rope actually forms.

1.3 Scope

In scope:

- A rope rendered in 3D in the browser, moved with the mouse. Touch is not required in the first version.
- A rope model with minimal tension, so the rope feels responsive. It keeps the rope from passing through itself or through an anchor.
- Recognising a named knot by its topology: two shapes are the same knot when one can be dragged into the other without passing through the rope.
- Three teaching modes: guided, sandbox, and quiz.
- The knots of sports climbing as a process. The priority areas are rappel, anchoring, belaying and cleaning. Prusik and other friction hitches are included for their shape only.

Deliberately out of scope:

- Load, holding and slipping behaviour. A knot that would hold or fail under force is not simulated.
- Engineering-accurate rope physics: strength, friction and wear.
- Cutting, splicing and rope joins that change the rope's topology.
- Fishing, sailing and other uses that are not climbing.
- Multiplayer, accounts and online play.
- Certification. Roppi is a proof of concept and a reference. Knots are for learning the shape, and a learner must practise them with a qualified expert. The product says so.

1.4 Who it is for

A complete beginner who wants to learn climbing knots before going on a wall, and who will practise them with an expert.

Chapter 2

Domain analysis

2.1 Concepts

Many climbing knots are tied *around* an anchor object or *join* two ropes. A rope alone cannot express them, so the domain has these concepts.

Rope A flexible curve of finite thickness, with two ends. It is the thing the player moves.

Anchor object A fixed object the rope can be tied around: a bar, a carabiner, a ring, or a tree. All anchor types are in the first version. Without it the topology of a hitch is incomplete.

Second rope A second rope, for bends that join two ropes.

Stopper end The free end of a rope in a rappel setup. A stopper keeps the rope from running through the device.

Knot One or more ropes together with their anchor objects, in a configuration the game recognises. A rope and its anchors count as one knot.

Stage A point in the climbing process: anchoring, belaying, rappel, cleaning, tie-in, or rescue. Each knot belongs to one or more stages.

Lesson A teaching unit: one knot, its stage, its guided steps, and its quiz.

2.2 Invariants

An invariant is a law that, if broken, means the game is no longer about climbing knots.

1. A rope cannot pass through itself or through an anchor object. A knot changes only when the rope is moved through a crossing or around an object.

2. A knot is identified by its topology, with its anchor objects included. Two configurations are the same knot when one can be dragged into the other without passing through the rope or an anchor. The same rope around a bar and around a carabiner can be different knots.

3. A bend joins two rope ends. Its ends must be the ends of the two ropes, not points in the middle. Mixing a bend and a hitch is an error.

4. A stopper knot is valid only at a free rope end. A middle overhand is not a stopper.

5. The rope keeps its length. Moving one part pulls the rest along.

6. A knot stays tied while nothing moves it. Only the player's input and the minimal tension change the shape.

7. Friction hitches are recognised by shape only. The game does not model how they grip under load.

2.3 Closure of open ends

A rope has two free ends, but a topological knot is a closed curve. For recognition, each free end is extended straight back, away from the viewer, to a point behind the scene, and the two extensions are joined there. The join is never drawn. Because it lies behind everything, any crossing it makes with the rope passes under the rope. The rope therefore looks open on screen and is closed for recognition.

2.4 Decided knot choices

Rappel stoppers: all options are taught. These are the double overhand, the triple overhand (barrel), and the overhand on a bight.

Joining two ropes for rappel: all options are taught. These are the double overhand and the flat overhand.

Figure-eight on a bight stopper: optional. The lesson notes that a stopper can be added, and does not require it.

2.5 Vocabulary

The book uses these terms, and every chunk name starts with them.

Strand A stretch of rope between two grab points, or between a grab point and an end.

Working end The free end the player is tying with.

Standing part The part of the rope that is not being worked.

Bight A U-shaped loop of rope.

Turn or wrap One full pass of rope around an anchor object or another part of the rope.

Hitch A knot that ties around an object.

Bend A knot that joins two ropes, or two ends of one rope.

Loop A knot that forms a closed loop in the rope.

Friction hitch A hitch that grips a rope under load and slides when unloaded.

Stopper A knot at a rope end that keeps the rope from running through a device.

Closure The way an open rope is closed for topology.

Tension A small pull that keeps the rope settled and responsive.

Chapter 3

Architecture

The system has these parts. Each one becomes a concept chapter later.

`React UI (Tailwind)` — lessons, modes, panels, quiz feedback. It updates on discrete events only, never on each frame.

`Rope model (Rapier)` — rigid links joined by joints.

`Renderer (three.js)` — one continuous tube, anchors and the frame. It is owned by one component.

`Snapshot` — a read-only copy of positions, passed to the topology.

`Topology` — closes the rope, projects it, builds a crossing code, and computes invariants.

`Recogniser` — matches the invariants against the knot library.

`Lessons` — data records that the three modes read.

3.1 Decision: rendering in three.js

We use three.js. The brief asks for it, and it runs in a browser with no install. The alternatives are a game engine such as Unity or Godot, which is heavier and not browser-first, and a 2D canvas, which cannot show the depth that the topology needs.

3.2 Decision: the rope is a Rapier chain, drawn as one tube

The rope is a chain of short rigid links joined neighbour to neighbour, using Rapier. The engine version is pinned at scaffold time.

The length of the rope is kept by a rope joint. This is a maximum-distance constraint, so it allows a little slack. Minimal tension is a weak pull toward a straight line.

The rope cannot pass through itself or through an anchor, because Rapier's contacts keep the links apart. Continuous collision detection is switched on for the links, so that a fast drag does not pass a link through another.

Each joint is created with contacts between its two links disabled, so neighbouring links never push on each other. Collision groups keep the links from touching other links where they should not.

The grab point is a kinematic body moved by the mouse. A kinematic body is not pushed by contacts, so the rope never blocks the player's hand.

Each frame, the link centres are smoothed with a Catmull-Rom curve. The tube's vertices are rewritten into a buffer that is allocated once. The rope therefore never looks like a set of links.

The alternatives considered were these. A custom position-based solver would need segment-to-segment collision written from scratch, which is the hardest part of the rope; Rapier already provides it, and its types confirm the joints and collision controls used here. A Rapier soft-body rope, built with `SoftBodyDesc.rope`, has self-contact options; it is the fallback if the chain proves unstable in stage one. Lines built with `Line2` or `MeshLine` are thin, and forum reports describe memory growth when they are updated each frame. Rebuilding `TubeGeometry` each frame costs an allocation each time.

Gate result. The spike measured the chain before the rest of stage one was built. With no limit, the tail of a forty-link rope dragged at 1.5 m/s reached 8 to 15 m/s, and the soft-body rope did the same. Both were rejected as unstable. The chain is kept with three settings: 32 solver iterations, linear damping of 0.5, and a link speed cap of 3 m/s applied after each physics step. With these, the stretch stays within 1.03 of the rope length, the figure-eight produces no overlap between links, no link enters the post, and a forty-link step costs about 0.7 ms. The cap is a deliberate departure from pure physics: at a hand speed of 1.5 m/s the tail sits at the cap, so the rope moves more slowly than the physics alone would move it. The soft-body rope is not used.

Rendering runs on demand. The frame loop runs while the rope moves and stops when it settles, as the three.js manual advises for scenes that change only on input.

3.3 Decision: closure is a point behind the background

For recognition only, the closure described in the analysis is used. The join lies behind everything, so the crossings it makes count as passing under the rope. This needs no special case in the topology code.

3.4 Decision: recognition by planar projection and invariants

The closed curve is projected onto the view plane. Each crossing and its over-or-under is read from depth, and the crossings are written as a Planar Diagram code. From that code two invariants are computed: the determinant, counted by Fox colouring, and the Alexander polynomial, from the Wirtinger presentation. A knot is matched against a library only when both invariants agree.

The recogniser reads a snapshot of the link positions. It never changes the rope or the physics world. It runs off the frame loop, throttled, and can move to a Web Worker if profiling shows it blocks frames.

The alternatives were these. Matching the pose against stored templates fails invariant 2, because the pose changes with every drag. The determinant alone cannot separate some knots.

The invariants do not separate every pair of knots, and they cannot tell a mirror image from its original. The library is therefore checked for collisions at build time, and the taught set is checked before lesson content is written.

3.5 Decision: anchors are closed components

Each anchor is a closed loop in space. A carabiner, a bar, a ring and a tree trunk are all loops. Anchors are separate components, so a hitch around an anchor is a link between the rope and the anchor. The recogniser handles several components in the same way as one.

Modelling an anchor as a fixed point cannot represent a rope wrapped around it. Modelling the anchor inside the rope merges two topologies, which is wrong.

3.6 Decision: lessons are data

Each knot is one data record. It holds the name, the stage, the web sources, the invariants, the guided steps and the quiz prompt. Guided, sandbox and quiz modes all read the same records. Quiz evaluation compares the recognised knot with the prompt. A hard-coded lesson would turn every

change into a code change, and a remote content service is not needed for a proof of concept.

3.7 Decision: literate source with weft

The source of truth is these `.weft` chapters. The `weft` tool tangles them into `src/`. We built weft 2.0.2 from its repository and copied the binary to the user's local bin directory.

3.8 Decision: tooling

Vite is the dev server and build tool. React is the UI. Tailwind styles the UI only, never the 3D canvas. Vitest runs the tests. three.js and Rapier are pinned to exact versions.

The 3D scene is owned by one component. That component creates the renderer, the scene and the physics world in an effect, and disposes of them on unmount.

The research behind the rendering choices comes from the three.js manual, which is in the three.js source repository. Its advice is these. Merging geometry reduces draw calls. Geometries, materials and textures are not freed automatically, so `dispose()` must be called when they are no longer used. A continuously animated loop is wasteful for a scene that changes only on input.

3.9 Forbidden edges

These are rules that a change must not break. They are checked in review.

1. The rope model does not import three.js.
2. The renderer never writes rope state.
3. The recogniser reads a snapshot and never changes the rope, the physics world or the scene.
4. Topology does not import React or three.js.
5. React holds no per-frame state. Per-frame data is written to refs.
6. Lesson records are data, with no code and no three.js or Rapier objects.
7. Anchors are scene data. The rope model does not own anchor geometry.
8. One component creates and disposes of the physics world and the renderer. No other code does.

9. Tailwind does not style the canvas.

3.10 Named seams

These contracts are empty until stage one.

RopeSnapshot The link centres, the radius and the two ends. Read-only.

AnchorSnapshot A closed loop of points, one per anchor.

KnotResult A name, or nothing if the knot is not recognised, together with its invariants.

LessonRecord The data described above.

Chapter 4

Development stages

Each stage is an edition of the project. At its boundary everything builds, the tests pass, and the stage's "proven when" fact holds. No stage begins before the previous one is proven.

4.1 Stage 1 — the walking skeleton

The thinnest slice that runs. It touches every stage-one concept, however thinly. The concepts are the rope model, one anchor (a bar), the closure, the topology (the determinant only), a two-entry recogniser (unknot and trefoil), the renderer, a React page with Tailwind and one lesson stub, and one lesson record. Tooling is Vite, Vitest and the pinned libraries.

Decision gate. Before the rest of stage one is built, the rigid chain is measured for stability, for no-pass behaviour, and for frame cost. The gate passed on 2026-10-19 with the rigid chain and a link speed cap. The soft-body rope was measured and not used. The measurements and settings are recorded in the rope decision of the architecture chapter.

Proven when the page runs; a rope can be dragged with the mouse and draws continuously; and a test that drives the rope into a trefoil returns `trefoil`, while a test of the unknot returns `unknot`.

Status. Built and tested. The page builds and runs in a headless browser, and the rope can be dragged. The trefoil test feeds the recogniser a scripted curve, not a rope tied by dragging. A check on a rope tied by hand is still open, and it belongs with the stage's last check.

4.2 Stage 2 — the recogniser

Adds the Alexander polynomial, the full library for the lessons, anchors of every type, and the build-time collision check.

Proven when every library entry is recognised from a scripted rope of that knot; every pair in the library is separated by the two invariants; and a clove hitch on a bar is recognised as a link with the bar.

4.3 Stage 3 — guided mode for one knot

Guided mode for overhand on a bight, with steps. The UI waits for the recogniser to confirm each step.

Proven when a guided lesson runs from its first step to the recognised knot, driven by the recogniser and not by a timer.

4.4 Stage 4 — sandbox

Free play. The recogniser names any library knot and says nothing otherwise.

Proven when a rope tied into each library knot is named in sandbox mode, and an unknown configuration is left unnamed.

4.5 Stage 5 — quiz mode

The instruction is shown, the player ties the knot, and asks for an evaluation.

Proven when a correct tie is marked correct; a wrong knot is marked wrong and named as the knot the player actually tied; and an untied rope is not evaluated.

4.6 Stage 6 — the knot catalogue

Adds the lessons from the research, in teaching order. Anchoring: figure-eight on a bight, clove hitch, girth hitch. Belaying: munter hitch, mule overhand. Rappel: the stoppers and the rope joins, as decided in the analysis. Cleaning: the rappel stoppers and backups, with no separate knot. Other: prusik and the friction hitches, shape only.

Proven when each lesson has a scripted reference rope that the recogniser names, and each lesson cites its web sources.

4.7 State of the work

Lifecycle: planned, drafted, implemented, stable.

Concept	Status	Stage	Chapter
Rope model	implemented	1	Rope model
Anchors	implemented	1 (bar), 2 (all)	Lessons and the page
Closure	implemented	1	Topology
Topology	implemented	1 (determinant), 2 (Alexander)	Topology
Recogniser	implemented	1 (two entries), 2 (library)	Topology
Renderer	implemented	1	Renderer
UI shell	implemented	1	Lessons and the page
Lesson record	implemented	1	Lessons and the page
Guided mode	planned	3	—
Sandbox mode	planned	4	—
Quiz mode	planned	5	—
Knot catalogue	planned	6	—

Chapter 5

Scaffold

Stage one needs a build tool, a test runner, a UI framework, and the two pinned libraries. Every version is exact. A rope that behaves today must behave the same after a dependency update, so nothing uses a caret or a tilde.

All code lives in the `app` directory, which the assembly routes to. The book stays at the root. The package manifest is written here and tangled out. The lock file is produced by npm when the packages are installed, and it is committed with the manifest.

5.1 Packages

Runtime: three.js renders, Rapier simulates, React draws the page. Development: Vite builds, Vitest tests, Tailwind styles the page (never the canvas), and TypeScript checks types. The versions were the latest releases at scaffold time.

⟨ *package manifest* 16 ⟩ ≡

```
{
  "name": "roppi",
  "version": "0.1.0",
  "private": true,
  "description": "a game of ropes using ldd",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "tsc --noEmit && vite build",
    "test": "vitest run",
    "typecheck": "tsc --noEmit"
  },
  "dependencies": {
    "@dimforge/rapier3d-compat": "0.21.0",
    "react": "19.3.1",
    "react-dom": "19.3.1",
```

```
    "three": "0.186.1"
  },
  "devDependencies": {
    "@tailwindcss/vite": "4.3.3",
    "@types/react": "19.3.0",
    "@types/react-dom": "19.3.0",
    "@types/three": "0.186.0",
    "@vitejs/plugin-react": "6.1.2",
    "tailwindcss": "4.3.3",
    "typescript": "5.9.3",
    "vite": "8.3.4",
    "vitest": "5.0.5"
  }
}
```

□

16 · used in 43a

5.2 Build and test configuration

Vite serves the page and builds it. The same file configures Vitest. Tests run in plain Node, because the topology and the rope model need no browser. The renderer is tested for its geometry, and the page itself is checked by running it.

⟨ *Vite configuration* 17a ⟩ ≡

```
import { defineConfig } from 'vitest/config';
import react from '@vitejs/plugin-react';
import tailwindcss from '@tailwindcss/vite';

export default defineConfig({
  plugins: [react(), tailwindcss()],
  test: {
    environment: 'node',
    include: ['src/**/*.test.ts'],
  },
});
```

□

17a · used in 43b

⟨ *TypeScript configuration* 17b ⟩ ≡

```
{
  "compilerOptions": {
    "target": "ES2022",
    "lib": ["ES2022", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "moduleResolution": "bundler",
    "jsx": "react-jsx",
    "strict": true,
```

```
    "noEmit": true,
    "isolatedModules": true,
    "skipLibCheck": true
  },
  "include": ["src", "vite.config.ts"]
}
```

□

17b · used in 43c

5.3 Page shell

The page has one root element. React mounts into it and nothing else is written into the document.

⟨ *HTML shell* 18a ⟩ ≡

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale
    =1.0" />
    <title>Roppi</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

□

18a · used in 43d

⟨ *Global styles* 18b ⟩ ≡

```
@import "tailwindcss";

html,
body,
#root {
  height: 100%;
  margin: 0;
}
```

□

18b · used in 44a

⟨ *Entry point* 18c ⟩ ≡

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
```

```
import { App } from ’./App’;
import ’./index.css’;

createRoot(document.getElementById(’root’)!).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

□

18c · used in 44b

Chapter 6

Rope model

The rope is a chain of forty links, each a small ball, joined neighbour to neighbour. Each joint keeps the links within one link spacing of each other, and a joint between neighbours turns off their contact so they never push each other. Everything else collides. The top link is held by a kinematic grab point that the player moves. The rope model knows nothing of three.js or React.

The numbers come from the decision gate in the architecture chapter. Solver iterations, damping and the speed cap are the values that passed it. The cap is applied after every physics step. It is a deliberate departure from pure physics, and the gate result records why.

6.1 Shapes the rest of the code reads

The rope model returns a snapshot: the link centres in order, from the grab end to the free end, and the link radius. Anchors are boxes in the scene. The model takes them as plain shapes, so it does not depend on what they are for.

⟨ *Rope: snapshot types* 20 ⟩ ≡

```
export interface Vec3 {
  x: number;
  y: number;
  z: number;
}

// The link centres in order, from the grab end to the free end.
export interface RopeSnapshot {
  readonly points: readonly Vec3[];
  readonly radius: number;
}

// An axis-aligned box given by its centre and its half sizes.
```

```
export interface Box {
  center: Vec3;
  half: Vec3;
}
```

□

20 · used in 44g

6.2 Physics model

The physics engine is loaded once and shared. A rope is created in a world of its own, so disposing of a rope frees exactly what it made.

The grab point is a kinematic body. A kinematic body is not pushed by contacts, so the rope never blocks the player's hand. Each link is a dynamic ball with continuous collision detection on, so a fast drag cannot pass a link through another. The speed cap is applied to every link after the step, and the step returns the fastest link speed, which the page uses to decide when the rope has settled.

⟨ *Rope: physics model* 21 ⟩ ≡

```
import RAPIER from '@dimforge/rapier3d-compat';
import type { Box, RopeSnapshot, Vec3 } from './snapshot';

export const LINK_COUNT = 40;
export const LINK_RADIUS = 0.02;
export const LINK_SPACING = 0.05;
export const MAX_LINK_SPEED = 3;
const SOLVER_ITERATIONS = 32;
const LINEAR_DAMPING = 0.5;
const GRAVITY = -9.81;

export interface Rope {
  grab(target: Vec3): void;
  step(): number;
  snapshot(): RopeSnapshot;
  dispose(): void;
}

let ready: Promise<void> | null = null;

export function initPhysics(): Promise<void> {
  ready ??= RAPIER.init();
  return ready;
}

export function createRope(top: Vec3, obstacles: readonly Box[]):
    Rope {
  const world = new RAPIER.World({ x: 0, y: GRAVITY, z: 0 });
  world.numSolverIterations = SOLVER_ITERATIONS;
```

```
const grab = world.createRigidBody(
  RAPIER.RigidBodyDesc.kinematicPositionBased().setTranslation(top
  .x, top.y, top.z),
);

const links: RAPIER.RigidBody[] = [];
for (let i = 0; i < LINK_COUNT; i++) {
  const body = world.createRigidBody(
    RAPIER.RigidBodyDesc.dynamic()
      .setTranslation(top.x, top.y - i * LINK_SPACING, top.z)
      .setLinearDamping(LINEAR_DAMPING)
      .setCcdEnabled(true),
  );
  world.createCollider(RAPIER.ColliderDesc.ball(LINK_RADIUS), body
  );
  links.push(body);
}

const origin = { x: 0, y: 0, z: 0 };
world.createImpulseJoint(RAPIER.JointData.spherical(origin, origin
  ), grab, links[0], true);
for (let i = 0; i < LINK_COUNT - 1; i++) {
  const joint = world.createImpulseJoint(
    RAPIER.JointData.rope(LINK_SPACING, origin, origin),
    links[i],
    links[i + 1],
    true,
  );
  joint.setContactsEnabled(false);
}

for (const box of obstacles) {
  const body = world.createRigidBody(
    RAPIER.RigidBodyDesc.fixed().setTranslation(box.center.x, box.
  center.y, box.center.z),
  );
  world.createCollider(RAPIER.ColliderDesc.cuboid(box.half.x, box.
  half.y, box.half.z), body);
}

return {
  grab(target) {
    grab.setNextKinematicTranslation(target);
  },
  step() {
    world.step();
    let fastest = 0;
    for (const body of links) {
      const v = body.linvel();
      const speed = Math.hypot(v.x, v.y, v.z);
      if (speed > MAX_LINK_SPEED) {
        const scale = MAX_LINK_SPEED / speed;
        body.setLinvel({ x: v.x * scale, y: v.y * scale, z: v.z *
  scale }, true);
      }
```

```
        fastest = Math.max(fastest, Math.min(speed, MAX_LINK_SPEED))
    ;
      }
      return fastest;
    },
    snapshot() {
      return { points: links.map((body) => body.translation()),
    radius: LINK_RADIUS };
    },
    dispose() {
      world.free();
    },
  };
}
```

□

21 · used in 45a

6.3 Tests

The tests carry the gate criteria forward, so a later change cannot quietly break the rope. A link may stretch by at most five per cent of the spacing. A drag must keep every link under the speed cap. A sweep past a bar must not put any link through the bar.

⟨ *Rope: tests* 23 ⟩ ≡

```
import { beforeAll, describe, expect, it } from 'vitest';
import { LINK_COUNT, LINK_RADIUS, LINK_SPACING, MAX_LINK_SPEED,
    createRope, initPhysics } from './rope';
import type { Vec3 } from './snapshot';

const TOP: Vec3 = { x: 0, y: 1.5, z: 0 };

// The largest neighbour distance, divided by the spacing. One means
     taut.
function longestStretch(points: readonly Vec3[]): number {
  let longest = 0;
  for (let i = 0; i < points.length - 1; i++) {
    const a = points[i];
    const b = points[i + 1];
    longest = Math.max(longest, Math.hypot(a.x - b.x, a.y - b.y, a.z
     - b.z) / LINK_SPACING);
  }
  return longest;
}

beforeAll(async () => {
  await initPhysics();
});

describe('rope model', () => {
```

```
  it('hangs from the grab with no link stretched beyond five per
    cent', () => {
    const rope = createRope(TOP, []);
    for (let s = 0; s < 300; s++) rope.step();
    const { points } = rope.snapshot();
    expect(points).toHaveLength(LINK_COUNT);
    expect(longestStretch(points)).toBeLessThan(1.05);
    rope.dispose();
  });

  it('keeps every link under the speed cap while dragged in a circle
    ', () => {
    const rope = createRope(TOP, []);
    for (let s = 0; s < 300; s++) rope.step();
    const radius = 0.4;
    const omega = 1.5 / radius;
    let fastest = 0;
    for (let s = 0; s < 360; s++) {
      const t = s / 60;
      rope.grab({ x: radius * Math.sin(omega * t), y: TOP.y, z:
    radius * (1 - Math.cos(omega * t)) });
      fastest = Math.max(fastest, rope.step());
    }
    expect(fastest).toBeLessThanOrEqual(MAX_LINK_SPEED);
    expect(longestStretch(rope.snapshot().points)).toBeLessThan
    (1.05);
    rope.dispose();
  });

  it('does not pass a link through a bar it is swept across', () =>
    {
    const bar = { center: { x: 0.3, y: 0, z: 0 }, half: { x: 0.01, y
    : 1, z: 0.01 } };
    const rope = createRope({ x: -0.1, y: 1.5, z: 0 }, [bar]);
    for (let s = 0; s < 120; s++) rope.step();
    let inside = 0;
    for (let s = 0; s < 120; s++) {
      rope.grab({ x: -0.1 + 0.9 * (s / 120), y: 1.5, z: 0 });
      rope.step();
      for (const p of rope.snapshot().points) {
        // A link centre inside the bar, grown by half a radius, is
    a pass-through.
        const margin = 0.01 + 0.5 * LINK_RADIUS;
        if (Math.abs(p.x - bar.center.x) < margin && Math.abs(p.z) <
     margin && Math.abs(p.y) < 1) inside++;
      }
    }
    expect(inside).toBe(0);
    rope.dispose();
  });
});
```

□

23 · used in 45b

Chapter 7

Topology

Recognition works on the shape of the rope, not on its pose. Three steps make that possible. The rope is closed, projected, and read as a knot diagram. Then one invariant is computed from the diagram. The determinant is the simplest, and it is the only one stage one uses.

7.1 Closure

A rope has two free ends, and a knot needs a closed loop. The two ends are joined through a point far behind the background. The joining path is never drawn. It is always behind the rope, so wherever it crosses the rope in the view, the rope lies over it. That is the physical picture of a rope lying in front of a wall, and it needs no special case in the code that follows.

⟨ *Topology: closure* 25 ⟩ ≡

```
import type { Vec3 } from '../rope/snapshot';

// How far behind the scene the joining path lies. Everything is
    closer than this.
const BEHIND = 1000;

// The rope's points, then two points far behind the last and first
    links. The
// loop closes from the second far point back to the first link.
export function closeRope(points: readonly Vec3[]): Vec3[] {
  const first = points[0];
  const last = points[points.length - 1];
  return [
    ...points,
    { x: last.x, y: last.y, z: -BEHIND },
    { x: first.x, y: first.y, z: -BEHIND },
  ];
}
```

□

7.2 Crossings and the Fox matrix

The viewer looks down the depth axis, so the diagram is the rope's projection onto the x and y plane. Two segments that cross in that plane form a crossing. The one with the greater depth is over. Depth is compared at the crossing point itself, not at the segment ends.

Every crossing has one under-passage, which splits the under-strand into arcs. Walking along the loop, arc j runs from under-passage j to under-passage $j + 1$. Each crossing touches three arcs: the arc entering its under-passage, the arc leaving it, and the arc that holds the over-strand. Fox colouring says that for every crossing, twice the over colour minus the two under colours is zero modulo the determinant. The coefficients of that condition form one row of the Fox matrix, and the determinant is read from that matrix.

⟨ *Topology: crossings and Fox matrix* 26 ⟩ ≡

```
import type { Vec3 } from '../rope/snapshot';

const EPS = 1e-12;

interface Crossing {
  overSegment: number;
  overAt: number;
  underSegment: number;
  underAt: number;
}

// Where segment a-b meets segment c-d in the view, as fractions
    along each, or
// null. Parallel or degenerate segments never meet.
function meet(a: Vec3, b: Vec3, c: Vec3, d: Vec3): { s: number; u:
    number } | null {
  const rx = b.x - a.x;
  const ry = b.y - a.y;
  const wx = d.x - c.x;
  const wy = d.y - c.y;
  const denom = rx * wy - ry * wx;
  if (Math.abs(denom) < EPS) return null;
  const qx = c.x - a.x;
  const qy = c.y - a.y;
  const s = (qx * wy - qy * wx) / denom;
  const u = (qx * ry - qy * rx) / denom;
  if (s < 0 || s >= 1 || u < 0 || u >= 1) return null;
  return { s, u };
}
```

```
// Every crossing of the closed loop in the view, with its over
    strand decided.
function crossingsOf(loop: readonly Vec3[]): Crossing[] {
  const n = loop.length;
  const found: Crossing[] = [];
  for (let i = 0; i < n; i++) {
    for (let j = i + 2; j < n; j++) {
      if (i === 0 && j === n - 1) continue; // these two segments
    share a vertex
      const a = loop[i];
      const b = loop[(i + 1) % n];
      const c = loop[j];
      const d = loop[(j + 1) % n];
      const hit = meet(a, b, c, d);
      if (!hit) continue;
      const depthI = a.z + hit.s * (b.z - a.z);
      const depthJ = c.z + hit.u * (d.z - c.z);
      if (depthI > depthJ) {
        found.push({ overSegment: i, overAt: hit.s, underSegment: j,
     underAt: hit.u });
      } else {
        found.push({ overSegment: j, overAt: hit.u, underSegment: i,
     underAt: hit.s });
      }
    }
  }
  return found;
}

// One row per crossing, one column per arc. Each row sums to zero.
export function foxMatrix(loop: readonly Vec3[]): number[][] {
  const found = crossingsOf(loop);
  const k = found.length;
  const underPositions = found.map((c) => c.underSegment + c.underAt
    );
  const order = found.map((_, index) => index).sort((p, q) =>
    underPositions[p] - underPositions[q]);
  const rank = new Array<number>(k);
  order.forEach((crossing, position) => {
    rank[crossing] = position;
  });
  const sortedUnder = order.map((crossing) => underPositions[
    crossing]);

  // The arc that holds a position along the loop: the last under-
    passage at or
  // before it starts that arc. Before the first passage, the arc
    wraps round.
  const arcAt = (position: number): number => {
    const passed = sortedUnder.filter((p) => p <= position).length;
    return (passed - 1 + k) % k;
  };

  const matrix = found.map(() => new Array<number>(k).fill(0));
  found.forEach((crossing, index) => {
```

```
    const j = rank[index];
    const row = matrix[index];
    row[(j - 1 + k) % k] -= 1;
    row[j] -= 1;
    row[arcAt(crossing.overSegment + crossing.overAt)] += 2;
  });
  return matrix;
}
```

□

26 · used in 45d

7.3 Determinant

The determinant of a knot is the absolute value of any one minor of the Fox matrix, the matrix with one row and one column removed. The integer determinant is computed exactly with the Bareiss method, which divides only where the result is known to be whole. A rope with no crossings has the empty matrix, whose determinant is one, which is the unknot's value.

⟨ *Topology: determinant* 28 ⟩ ≡

```
// Exact integer determinant by the Bareiss method. The empty matrix
     has determinant one.
export function bareiss(input: readonly (readonly number[])[]):
    number {
  const m = input.map((row) => [...row]);
  const n = m.length;
  let sign = 1;
  let previous = 1;
  for (let k = 0; k < n - 1; k++) {
    if (m[k][k] === 0) {
      const swap = m.findIndex((row, i) => i > k && row[k] !== 0);
      if (swap === -1) return 0;
      [m[k], m[swap]] = [m[swap], m[k]];
      sign = -sign;
    }
    for (let i = k + 1; i < n; i++) {
      for (let j = k + 1; j < n; j++) {
        m[i][j] = (m[i][j] * m[k][k] - m[i][k] * m[k][j]) / previous
    ;
      }
    }
    previous = m[k][k];
  }
  return n === 0 ? 1 : sign * m[n - 1][n - 1];
}

// The knot determinant: the absolute value of the Fox matrix with
    its last row and column removed.
export function knotDeterminant(matrix: readonly (readonly number[])
    []): number {
```

```
  const k = matrix.length;
  const minor = matrix.slice(0, k - 1).map((row) => row.slice(0, k -
    1));
  return Math.abs(bareiss(minor));
}
```

□

28 · used in 45e

7.4 Recogniser

The recogniser closes the rope, builds the Fox matrix, takes the determinant, and looks the value up in the library. The library has two entries in stage one. A determinant that matches no entry gives no name, which is the honest answer until the library grows. It does not guess.

The recogniser reads a snapshot and changes nothing. It is a pure function, so the page can call it whenever the rope has moved.

⟨ *Topology: recogniser* 29 ⟩ ≡

```
import type { RopeSnapshot } from '../rope/snapshot';
import { closeRope } from './closure';
import { foxMatrix } from './crossings';
import { knotDeterminant } from './determinant';

export interface KnotEntry {
  name: string;
  determinant: number;
}

// Stage one knows two knots. Stage two adds the Alexander
    polynomial and the full library.
export const LIBRARY: readonly KnotEntry[] = [
  { name: 'unknot', determinant: 1 },
  { name: 'trefoil', determinant: 3 },
];

export interface KnotResult {
  name: string | null;
  determinant: number;
  crossings: number;
}

export function recognise(snapshot: RopeSnapshot): KnotResult {
  const matrix = foxMatrix(closeRope(snapshot.points));
  const determinant = knotDeterminant(matrix);
  const entry = LIBRARY.find((known) => known.determinant ===
    determinant);
  return { name: entry?.name ?? null, determinant, crossings: matrix
    .length };
}
```

□

29 · used in 45f

7.5 Tests

The tests check the algorithm on shapes whose answers are known. A straight rope and a rope with one curl are unknots. The standard trefoil curve is a trefoil, and so is its mirror image, because the determinant cannot tell a knot from its mirror. The Fox matrix of a two-by-two case gives a determinant of three, which checks the exact arithmetic itself.

⟨ *Topology: tests* 30 ⟩ ≡

```
import { describe, expect, it } from 'vitest';
import type { Vec3 } from '../rope/snapshot';
import { bareiss } from './determinant';
import { recognise } from './recogniser';

const SAMPLES = 120;

// Sample a closed curve at evenly spaced parameter values.
function sample(curve: (t: number) => Vec3): Vec3[] {
  return Array.from({ length: SAMPLES }, (_, i) => curve((2 * Math.
    PI * i) / SAMPLES));
}

const trefoil = (t: number): Vec3 => ({
  x: Math.sin(t) + 2 * Math.sin(2 * t),
  y: Math.cos(t) - 2 * Math.cos(2 * t),
  z: -Math.sin(3 * t),
});

const mirroredTrefoil = (t: number): Vec3 => ({ ...trefoil(t), z: -
    trefoil(t).z });

// A limacon, which crosses itself once in the view. Its depth makes
     one strand pass over the other.
const curl = (t: number): Vec3 => {
  const r = 0.5 + Math.cos(t);
  return { x: r * Math.cos(t), y: r * Math.sin(t), z: 0.2 * Math.sin
    (t) };
};

describe('determinant', () => {
  it('computes exact determinants of small integer matrices', () =>
    {
    expect(bareiss([[2, -1], [-1, 2]])).toBe(3);
    expect(bareiss([[1, 0, 0], [0, 1, 0], [0, 0, 1]])).toBe(1);
    expect(bareiss([[0, 1], [1, 0]])).toBe(-1);
    expect(bareiss([])).toBe(1);
  });
```

```
});

describe('recogniser', () => {
  it('names a straight rope as the unknot', () => {
    const points = Array.from({ length: 40 }, (_, i) => ({ x: i *
    0.05, y: 0, z: 0 }));
    const result = recognise({ points, radius: 0.02 });
    expect(result.name).toBe('unknot');
    expect(result.crossings).toBe(0);
  });

  it('names a rope with one curl as the unknot', () => {
    const result = recognise({ points: sample(curl), radius: 0.02 })
    ;
    expect(result.name).toBe('unknot');
    expect(result.determinant).toBe(1);
  });

  it('names the trefoil curve as the trefoil', () => {
    const result = recognise({ points: sample(trefoil), radius: 0.02
     });
    expect(result.name).toBe('trefoil');
    expect(result.determinant).toBe(3);
    expect(result.crossings).toBe(3);
  });

  it('names the mirror image of the trefoil as the trefoil', () => {
    const result = recognise({ points: sample(mirroredTrefoil),
    radius: 0.02 });
    expect(result.name).toBe('trefoil');
  });
});
```

□

30 · used in 45g

Chapter 8

Renderer

The rope is drawn as one continuous tube, not as a row of spheres or a line. The tube follows a smooth curve through the link centres, so it reads as a rope. The renderer owns no physics. It reads a snapshot and writes geometry.

The tube's vertex buffer is allocated once, when the renderer is built. Each frame the same buffer is rewritten in place, so the frame loop allocates no new geometry. That follows the three.js manual, which says a geometry is not freed automatically and that a buffer should be reused when its layout is fixed. Three.js itself allocates small temporaries when it evaluates the curve, but it does not reallocate the buffer.

8.1 The tube

Each frame the link centres become the control points of a Catmull-Rom curve. Along the curve the renderer places a ring of vertices, perpendicular to the tangent. The ring's orientation is carried from one ring to the next by projecting the previous normal onto the new tangent plane. This keeps the tube from twisting as the curve bends. The first ring starts from an arbitrary perpendicular.

The buffer's layout depends only on the link count and the radial resolution, both fixed at construction. The index buffer is built once and never changes.

⟨ *Renderer: rope tube* 32 ⟩ ≡

```
import * as THREE from 'three';
import type { RopeSnapshot } from '../rope/snapshot';

const RADIAL = 8;
const SAMPLES_PER_LINK = 4;
```

```
// One tube along the rope. The buffers are allocated once in the
    constructor, and
// update() rewrites the vertex positions in place.
export class RopeTube {
  readonly mesh: THREE.Mesh;
  readonly geometry = new THREE.BufferGeometry();
  private readonly material = new THREE.MeshStandardMaterial({
    color: 0xd9a441,
    roughness: 0.6,
    side: THREE.DoubleSide,
  });
  private readonly positions: Float32Array;
  private readonly attribute: THREE.BufferAttribute;
  private readonly controls: THREE.Vector3[];
  private readonly curve: THREE.CatmullRomCurve3;
  private readonly rings: number;
  private readonly radius: number;
  private readonly center = new THREE.Vector3();
  private readonly tangent = new THREE.Vector3();
  private readonly normal = new THREE.Vector3();
  private readonly binormal = new THREE.Vector3();
  private readonly seed = new THREE.Vector3();

  constructor(linkCount: number, radius: number) {
    this.radius = radius;
    this.rings = (linkCount - 1) * SAMPLES_PER_LINK + 1;

    this.positions = new Float32Array(this.rings * (RADIAL + 1) * 3)
    ;
    this.attribute = new THREE.BufferAttribute(this.positions, 3);
    this.geometry.setAttribute('position', this.attribute);

    const indices: number[] = [];
    for (let r = 0; r < this.rings - 1; r++) {
      for (let s = 0; s < RADIAL; s++) {
        const a = r * (RADIAL + 1) + s;
        const c = a + (RADIAL + 1);
        indices.push(a, c, a + 1, a + 1, c, c + 1);
      }
    }
    this.geometry.setIndex(indices);

    this.controls = Array.from({ length: linkCount }, () => new
    THREE.Vector3());
    this.curve = new THREE.CatmullRomCurve3(this.controls);
    this.mesh = new THREE.Mesh(this.geometry, this.material);
    // The bounds are not recomputed each frame, so culling would
    use stale ones.
    this.mesh.frustumCulled = false;
  }

  update(snapshot: RopeSnapshot): void {
    snapshot.points.forEach((p, i) => this.controls[i].set(p.x, p.y,
     p.z));
    this.curve.points = this.controls;
```

```
    for (let r = 0; r < this.rings; r++) {
      const t = r / (this.rings - 1);
      this.curve.getPoint(t, this.center);
      this.curve.getTangent(t, this.tangent);
      if (r === 0) {
        // Start from any direction that is not along the tangent.
        this.seed.set(Math.abs(this.tangent.x) > 0.9 ? 0 : 1, Math.
    abs(this.tangent.x) > 0.9 ? 1 : 0, 0);
        this.normal.copy(this.seed);
      }
      // Project the carried normal onto the plane perpendicular to
    the tangent.
      this.normal.addScaledVector(this.tangent, -this.normal.dot(
    this.tangent)).normalize();
      this.binormal.crossVectors(this.tangent, this.normal);

      for (let s = 0; s <= RADIAL; s++) {
        const angle = (s / RADIAL) * Math.PI * 2;
        const cos = Math.cos(angle) * this.radius;
        const sin = Math.sin(angle) * this.radius;
        const o = (r * (RADIAL + 1) + s) * 3;
        this.positions[o] = this.center.x + this.normal.x * cos +
    this.binormal.x * sin;
        this.positions[o + 1] = this.center.y + this.normal.y * cos
    + this.binormal.y * sin;
        this.positions[o + 2] = this.center.z + this.normal.z * cos
    + this.binormal.z * sin;
      }
    }
    this.attribute.needsUpdate = true;
  }

  dispose(): void {
    this.geometry.dispose();
    this.material.dispose();
  }
}
```

□

32 · used in 46a

8.2 Tests

The renderer is tested without a browser. A straight rope is the simplest case: every vertex must lie exactly one radius from the rope's axis. The test also checks that the buffer is the same object before and after an update, which is the no-allocation promise made above.

⟨ *Renderer: tube tests* 34 ⟩ ≡

```
import { describe, expect, it } from 'vitest';
```

```
import type { RopeSnapshot } from '../rope/snapshot';
import { RopeTube } from './ropeTube';

const LINKS = 40;
const RADIUS = 0.02;

const straight: RopeSnapshot = {
  radius: RADIUS,
  points: Array.from({ length: LINKS }, (_, i) => ({ x: 0, y: -i *
    0.05, z: 0 })),
};

describe('rope tube', () => {
  it('keeps the same vertex buffer across updates', () => {
    const tube = new RopeTube(LINKS, RADIUS);
    const before = tube.geometry.getAttribute('position').array;
    tube.update(straight);
    tube.update(straight);
    expect(tube.geometry.getAttribute('position').array).toBe(before
    );
    tube.dispose();
  });

  it('places every vertex one radius from the rope axis', () => {
    const tube = new RopeTube(LINKS, RADIUS);
    tube.update(straight);
    const array = tube.geometry.getAttribute('position').array;
    for (let i = 0; i < array.length; i += 3) {
      const distance = Math.hypot(array[i], array[i + 2]);
      expect(distance).toBeCloseTo(RADIUS, 6);
    }
    tube.dispose();
  });
});
```

□

34 · used in 46b

Chapter 9

Lessons and the page

The page is React and Tailwind. It holds the lesson text and the recognised name, and it does not hold any per-frame state. The scene, with the physics world, the renderer and the rope tube, lives in one component, which creates them when it mounts and disposes of them when it unmounts.

9.1 Lesson record

A lesson is data. It names the knot, the stage that teaches it, its web sources, the invariants a recogniser must match, and the prompt the player reads. Stage one has one record, the overhand knot, so the page has something to show. The guided, sandbox and quiz modes arrive in later stages and read the same records.

The source is the Knot Atlas entry for 3_1, which the research chapter records as confirmed. The determinant is three, as the topology chapter computes.

⟨ *Lessons: record and stub* 36 ⟩ ≡

```
export interface Invariants {
  determinant: number;
}

export interface LessonRecord {
  id: string;
  name: string;
  stage: number;
  sources: readonly string[];
  invariants: Invariants;
  prompt: string;
}

export const STAGE_ONE_LESSONS: readonly LessonRecord[] = [
  {
    id: 'overhand',
```

```
    name: 'Overhand knot',
    stage: 1,
    sources: ['https://katlas.org/wiki/3_1'],
    invariants: { determinant: 3 },
    prompt: 'Tie an overhand knot in the rope. The recogniser names
    it when it is tied.',
  },
];
```

□

36 · used in 44e

9.2 Anchor

The bar is the one anchor in stage one. It is scene data: a box with a centre and half sizes. The rope model receives it as a plain shape, and the renderer draws it as a mesh. Neither owns it. The bar lies along the depth axis, so a rope that is dragged across it in the view can drape over it, and the rope cannot pass through it.

⟨ *Anchors: bar* 37a ⟩ ≡

```
import type { Box } from '../rope/snapshot';

export const BAR: Box = {
  center: { x: 0.6, y: 0.2, z: 0 },
  half: { x: 0.02, y: 0.02, z: 0.4 },
};
```

□

37a · used in 44f

9.3 Rope stage

The stage component owns the whole scene. Three effects keep it honest.

The physics loop runs only while the rope moves. When the fastest link is slower than the settle speed for thirty frames, the loop stops, and a pointer event starts it again. That is render on demand, as the architecture chapter asks.

Recognition runs at most four times a second while the rope moves, and once when it settles. It sets state only when the name changes, so React re-renders on a change of result, never on each frame.

Pointer input moves the grab point in the plane facing the viewer. The wheel moves it in depth, because a knot needs depth, and a drag in the plane alone produces none.

⟨ *UI: rope stage* 37b ⟩ ≡

```
import { useEffect, useRef, useState } from 'react';
import * as THREE from 'three';
import { RopeTube } from '../render/ropeTube';
import { BAR } from '../anchors/bar';
import { LINK_COUNT, LINK_RADIUS, createRope, initPhysics, type Rope
    } from '../rope/rope';
import { recognise } from '../topology/recogniser';

const TOP = { x: 0, y: 1.2, z: 0 };
const SETTLE_SPEED = 0.01;
const SETTLE_FRAMES = 30;
const RECOGNISE_EVERY_MS = 250;
const DEPTH_STEP = 0.002;

export function RopeStage() {
  const hostRef = useRef<HTMLDivElement>(null);
  const [name, setName] = useState<string | null>(null);

  useEffect(() => {
    const host = hostRef.current;
    if (!host) return;

    let disposed = false;
    let frame = 0;
    let running = false;
    let rope: Rope | null = null;
    let tube: RopeTube | null = null;
    let renderer: THREE.WebGLRenderer | null = null;
    let barGeometry: THREE.BoxGeometry | null = null;
    let barMaterial: THREE.MeshStandardMaterial | null = null;
    let still = 0;
    let lastRecognised = 0;
    let grabZ = 0;
    let cleanup = () => {};
    const target = { x: TOP.x, y: TOP.y, z: TOP.z };

    const raycaster = new THREE.Raycaster();
    const pointer = new THREE.Vector2();
    const plane = new THREE.Plane();
    const hit = new THREE.Vector3();
    let dragging = false;

    initPhysics().then(() => {
      if (disposed) return;

      rope = createRope(TOP, [BAR]);
      tube = new RopeTube(LINK_COUNT, LINK_RADIUS);

      const scene = new THREE.Scene();
      scene.background = new THREE.Color(0x0f172a);
      scene.add(tube.mesh);
      scene.add(new THREE.AmbientLight(0xffffff, 0.6));
      const light = new THREE.DirectionalLight(0xffffff, 1.2);
      light.position.set(1, 2, 3);
```

```
  scene.add(light);

  barGeometry = new THREE.BoxGeometry(BAR.half.x * 2, BAR.half.y
 * 2, BAR.half.z * 2);
  barMaterial = new THREE.MeshStandardMaterial({ color: 0x64748b
 });
  const barMesh = new THREE.Mesh(barGeometry, barMaterial);
  barMesh.position.set(BAR.center.x, BAR.center.y, BAR.center.z)
;
  scene.add(barMesh);

  const camera = new THREE.PerspectiveCamera(40, host.
clientWidth / host.clientHeight, 0.01, 100);
  camera.position.set(0, 0.2, 4);
  camera.lookAt(0, 0.2, 0);

  renderer = new THREE.WebGLRenderer({ antialias: true });
  renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2));
  renderer.setSize(host.clientWidth, host.clientHeight);
  host.appendChild(renderer.domElement);

  const canvas = renderer.domElement;
  const draw = renderer;
  const world = rope;
  const strip = tube;

  const animate = () => {
    const fastest = world.step();
    const snapshot = world.snapshot();
    strip.update(snapshot);
    draw.render(scene, camera);

    still = fastest < SETTLE_SPEED ? still + 1 : 0;
    const now = performance.now();
    if (still === SETTLE_FRAMES || (still === 0 && now -
lastRecognised > RECOGNISE_EVERY_MS)) {
      lastRecognised = now;
      setName(recognise(snapshot).name);
    }
    if (still < SETTLE_FRAMES) {
      frame = requestAnimationFrame(animate);
    } else {
      running = false;
    }
  };

  const wake = () => {
    still = 0;
    if (!running) {
      running = true;
      frame = requestAnimationFrame(animate);
    }
  };

  const moveGrab = (event: PointerEvent) => {
```

```
    const rect = canvas.getBoundingClientRect();
    pointer.set(
      ((event.clientX - rect.left) / rect.width) * 2 - 1,
      -((event.clientY - rect.top) / rect.height) * 2 + 1,
    );
    raycaster.setFromCamera(pointer, camera);
    plane.set(new THREE.Vector3(0, 0, 1), -grabZ);
    if (raycaster.ray.intersectPlane(plane, hit)) {
      target.x = hit.x;
      target.y = hit.y;
      target.z = grabZ;
      world.grab(target);
      wake();
    }
  };

  const onDown = (event: PointerEvent) => {
    dragging = true;
    canvas.setPointerCapture(event.pointerId);
    moveGrab(event);
  };
  const onMove = (event: PointerEvent) => {
    if (dragging) moveGrab(event);
  };
  const onUp = () => {
    dragging = false;
  };
  const onWheel = (event: WheelEvent) => {
    grabZ = Math.max(-1, Math.min(1, grabZ - event.deltaY *
DEPTH_STEP));
    target.z = grabZ;
    world.grab(target);
    wake();
  };
  const onResize = () => {
    camera.aspect = host.clientWidth / host.clientHeight;
    camera.updateProjectionMatrix();
    draw.setSize(host.clientWidth, host.clientHeight);
  };

  canvas.addEventListener('pointerdown', onDown);
  canvas.addEventListener('pointermove', onMove);
  canvas.addEventListener('pointerup', onUp);
  canvas.addEventListener('pointercancel', onUp);
  canvas.addEventListener('wheel', onWheel);
  window.addEventListener('resize', onResize);

  cleanup = () => {
    canvas.removeEventListener('pointerdown', onDown);
    canvas.removeEventListener('pointermove', onMove);
    canvas.removeEventListener('pointerup', onUp);
    canvas.removeEventListener('pointercancel', onUp);
    canvas.removeEventListener('wheel', onWheel);
    window.removeEventListener('resize', onResize);
  };
```

```
      wake();
    });

    return () => {
      disposed = true;
      cancelAnimationFrame(frame);
      cleanup();
      rope?.dispose();
      tube?.dispose();
      barGeometry?.dispose();
      barMaterial?.dispose();
      if (renderer) {
        renderer.dispose();
        renderer.domElement.remove();
      }
    };
  }, []);

  return (
    <div className="relative h-full w-full">
      <div ref={hostRef} className="absolute inset-0" />
      <p className="pointer-events-none absolute bottom-3 left-3
    rounded bg-slate-900/80 px-2 py-1 text-sm text-slate-200">
        Recognised: {name ?? 'nothing yet'}
      </p>
    </div>
  );
}
```

□

37b · used in 44d

9.4 Application

The application is one screen: the lesson's name and prompt at the top, the rope stage filling the rest. The stage is the only part with per-frame work.

⟨ *UI: application* 41 ⟩ ≡

```
import { STAGE_ONE_LESSONS } from './lessons/record';
import { RopeStage } from './ui/RopeStage';

export function App() {
  const lesson = STAGE_ONE_LESSONS[0];
  return (
    <main className="flex h-full flex-col bg-slate-950 text-slate
    -100">
      <header className="border-b border-slate-800 px-4 py-3">
        <h1 className="text-lg font-semibold">Roppi</h1>
        <p className="text-sm text-slate-400">
          {lesson.name}. {lesson.prompt}
        </p>
```

```
      </header>
      <section className="relative min-h-0 flex-1">
        <RopeStage />
      </section>
    </main>
  );
}
```

□

41 · used in 44c

Chapter 10

Assembly

This is the only file that will carry `@o` directives. The concept chapters never name an output path. They define chunks, and this file routes those chunks to the files the build expects, so the layout can change without touching a single narrative.

Stage one routes the scaffold, the rope model, the topology, the renderer, and the page. Each route below names one file and the chunk that fills it.

10.1 Tooling and page

⟨ `app/package.json` 43a ⟩ ≡

⟨ *package manifest* 16 ⟩

□

43a

⟨ `app/vite.config.ts` 43b ⟩ ≡

⟨ *Vite configuration* 17a ⟩

□

43b

⟨ `app/tsconfig.json` 43c ⟩ ≡

⟨ *TypeScript configuration* 17b ⟩

□

43c

⟨ `app/index.html` 43d ⟩ ≡

⟨ *HTML shell* 18a ⟩

□

43d

⟨ app/src/index.css 44a ⟩ ≡

⟨ *Global styles* 18b ⟩

□

44a

⟨ app/src/main.tsx 44b ⟩ ≡

⟨ *Entry point* 18c ⟩

□

44b

⟨ app/src/App.tsx 44c ⟩ ≡

⟨ *UI: application* 41 ⟩

□

44c

⟨ app/src/ui/RopeStage.tsx 44d ⟩ ≡

⟨ *UI: rope stage* 37b ⟩

□

44d

⟨ app/src/lessons/record.ts 44e ⟩ ≡

⟨ *Lessons: record and stub* 36 ⟩

□

44e

⟨ app/src/anchors/bar.ts 44f ⟩ ≡

⟨ *Anchors: bar* 37a ⟩

□

44f

10.2 Rope model

⟨ app/src/rope/snapshot.ts 44g ⟩ ≡

⟨ *Rope: snapshot types* 20 ⟩

□

44g

⟨ app/src/rope/rope.ts 45a ⟩ ≡

⟨ *Rope: physics model* 21 ⟩

□

45a

⟨ app/src/rope/rope.test.ts 45b ⟩ ≡

⟨ *Rope: tests* 23 ⟩

□

45b

10.3 Topology

⟨ app/src/topology/closure.ts 45c ⟩ ≡

⟨ *Topology: closure* 25 ⟩

□

45c

⟨ app/src/topology/crossings.ts 45d ⟩ ≡

⟨ *Topology: crossings and Fox matrix* 26 ⟩

□

45d

⟨ app/src/topology/determinant.ts 45e ⟩ ≡

⟨ *Topology: determinant* 28 ⟩

□

45e

⟨ app/src/topology/recogniser.ts 45f ⟩ ≡

⟨ *Topology: recogniser* 29 ⟩

□

45f

⟨ app/src/topology/topology.test.ts 45g ⟩ ≡

⟨ *Topology: tests* 30 ⟩

□

45g

10.4 Renderer

⟨ app/src/render/ropeTube.ts 46a ⟩ ≡

⟨ *Renderer: rope tube* 32 ⟩

□

46a

⟨ app/src/render/ropeTube.test.ts 46b ⟩ ≡

⟨ *Renderer: tube tests* 34 ⟩

□

46b